

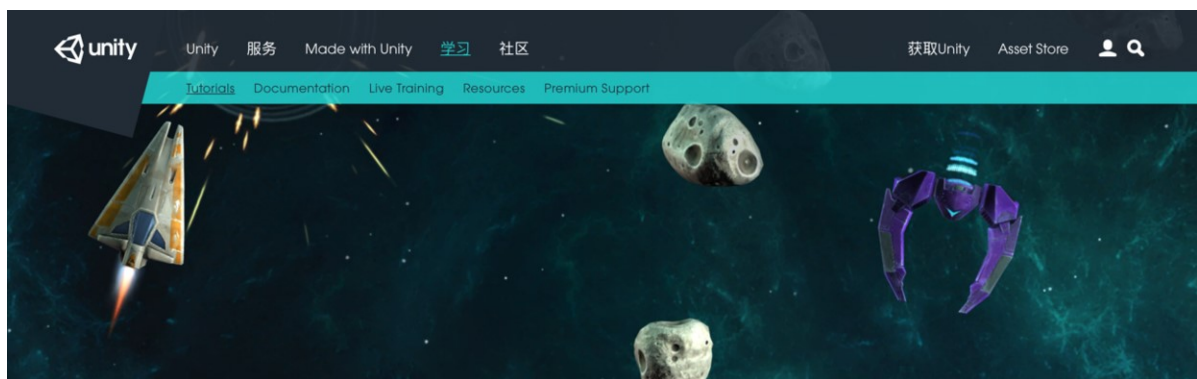
Unity官方实例教程 Space Shooter (二)

笔记本： 网页裁剪

创建时间： 2017/6/25 22:25

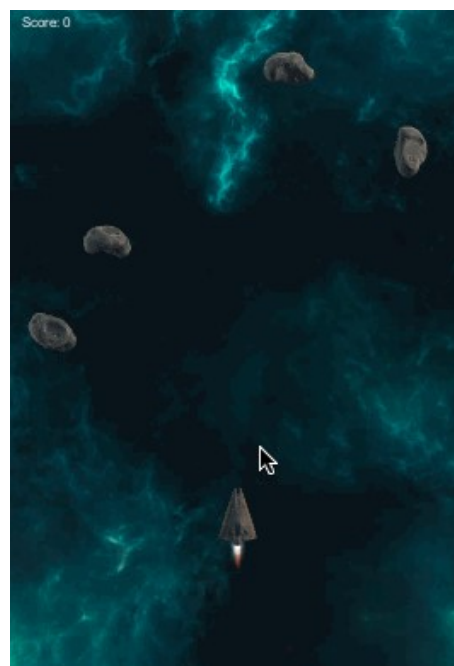
更新时间： 2017/6/26 3:23

Unity官方实例教程 Space Shooter (二)



Space Shooter 太空射击

前言



游戏截图

在[Unity官方实例教程 Space Shooter \(一\)](#)中，我们学会了：

- 如何下载和导入资源
- 如何设置游戏视窗大小
- 添加了飞船和飞船喷射效果
- Capsule Collider和Mesh Collider的用法

但是我们还没有为游戏添加任何的操作，现在还只是一个可以看，但不能玩的版本，所以今天我就让游戏可以玩起来！

你将学到什么？

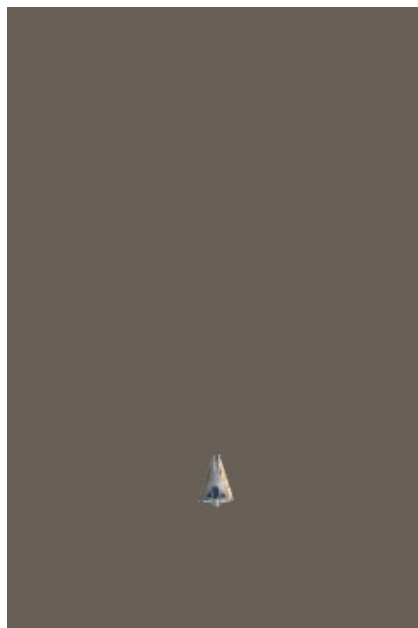
- 如何控制飞船移动
- 如何限制飞船的移动范围
- 如何整理公共变量
- 添加游戏背景
- 如何创建一个新材质
- 如何让飞机发射子弹

一、调整摄像机的位置

我们进入**Game**窗口，可以看到现在飞船是出于屏幕的正中间的，对于我们最终的游戏来说，这不是一个合适的位置，所以我们首先调整一下摄像机的位置，来改变飞船在屏幕中的位置

这边其实也可以调整飞船的位置，来达到同样的目的，但是让飞船处于 $(0, 0, 0)$ 的位置，会利于我们之后的制作，所以本例中我们采用调整摄像机的位置的方法来实现

我们选中**Main Camera**，然后将其的**Position**的**Z**值调整为**5**，然后我们看一下飞船的位置，如下：



这是一个不错的位置，接下来我们便可以来实现飞船的移动功能了

二、控制飞船移动

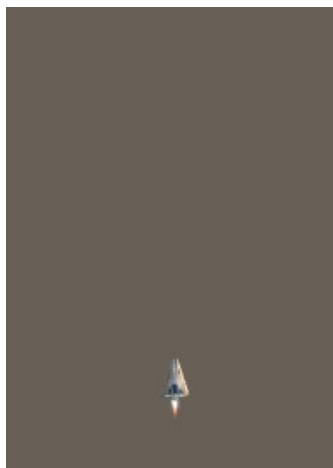
为了实现飞船的移动，首先我们给飞船也就是我们的**Player**添加一个脚本（Script），关于如何添加脚本请参见[Unity官方实例教程 Roll-a-Ball（一）](#)，在这里我们将脚本的名字命名为**PlayerController**，然后我打开脚本，添加以下代码：

```
4 public class PlayerController : MonoBehaviour {
5     public float speed;
6
7     // Use this for initialization
8     void Start () {
9
10    }
11
12    // Update is called once per frame
13    void Update () {
14
15    }
16
17    void FixedUpdate() {
18        float moveHorizontal = Input.GetAxis("Horizontal");
19        float moveVertical = Input.GetAxis("Vertical");
20
21        Vector3 movement = new Vector3 (moveHorizontal, 0.0f, moveVertical);
22        GetComponent<Rigidbody> ().velocity = movement * speed;
23    }
24 }
```

完整的代码

- 首先我们定义了公共的变量**speed**
- 然后用两个变量**moveHorizontal**和**moveVertical**记录玩家输入的方向数据
- 然后我们新建一个**Vector3**类型的变量**movement**，并且将**moveHorizontal**和**moveVertical**分别赋值给**movement**的X和Z
- 最后我们获取Player的刚体的速度属性，将**movement**乘上**speed**的值赋给它

代码写完之后，我们保存一下代码，然后回到Unity编辑器，将**speed**设置为**10**，接着运行游戏来检测我们刚刚的代码是否起到作用了。如无意外，现在我们可以控制飞船上下左右移动了！



飞船移动

不过现在飞船移动好像不是那么的生动，我们在给飞船左右移动的时候，让飞船整个身体倾斜一点，这样更加性感，我们再次打开脚本，增加以下代码：

```
using class PlayerController : MonoBehaviour {
    public float speed;
    public float tilt;

    // Use this for initialization
    void Start () {

    }

    // Update is called once per frame
    void Update () {
    }

    void FixedUpdate() {
        float moveHorizontal = Input.GetAxis("Horizontal");
        float moveVertical = Input.GetAxis("Vertical");

        Vector3 movement = new Vector3 (moveHorizontal, 0.0f, moveVertical);
        GetComponent<Rigidbody> ().velocity = movement * speed;

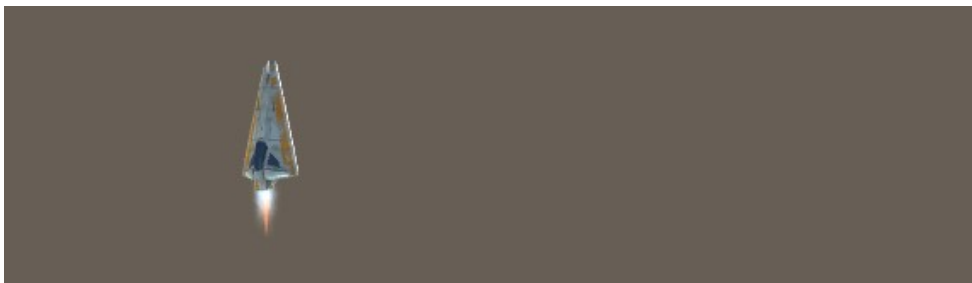
        GetComponent<Rigidbody> ().rotation = Quaternion.Euler (0.0f,
                                                                0.0f,
                                                                GetComponent<Rigidbody> ().velocity.x * -tilt);
    }
}
```

完整的代码

- 首先我们添加一个公共变量**tilt**，来设置飞船倾斜的幅度
- 然后我们获取**Player**刚体属性的rotation，将我们移动的速度x值乘上-tilt赋值给rotation属性

关于**Quaternion**的详细信息有兴趣的朋友可以去查阅官方文档，后面会出专门的文章来详细讲解，这边就不做详细介绍了，大家只需要知道这句话的作用就是实现飞船左右移动时的倾斜功能

代码写完之后，我们保存一下，然后回到Unity编辑器，将**tilt**设置为**5**，接着运行游戏，我们就可以检查刚刚的代码是否起到作用，如无意外，飞船左右移动的时候就会有倾斜效果了



倾斜效果

三、限制飞船的移动范围

我们实现了飞船移动功能后，会发现一个问题，就是飞船可以移动到屏幕外面去，这样玩家就看不

到飞船了，这不是好的体验，就像与航天联一样，所以我们需要限制一下飞船的移动范围，让它无法飞出我们的屏幕外

让我们来思考一下，如何来限制飞船的移动范围呢？首先我们飞船的移动范围，换句话说就是飞船的**Position**属性，而它的范围呢，就是我们的屏幕边界，由于我们游戏视角是俯视角，飞船的Y轴位置不会变，所以我们只需限制飞船在X和Z轴上的位置即可，我们在编辑器拖动飞船到屏幕的最左边，可以看到飞船的X大约在-6左右，我们使用同样的方法，可以测试飞船在X和Z轴上的范围分别是(-6, 6)和(-3, 8)。

接下来我们看看如何在代码中来限制飞船的位置：

```
System.Serializable]
public class Boundary {
    public float xMin, xMax, zMin, zMax;

public class PlayerController : MonoBehaviour {
    public float speed;
    public float tilt;
    public Boundary boundary;

    void FixedUpdate() {
        float moveHorizontal = Input.GetAxis("Horizontal");
        float moveVertical = Input.GetAxis("Vertical");

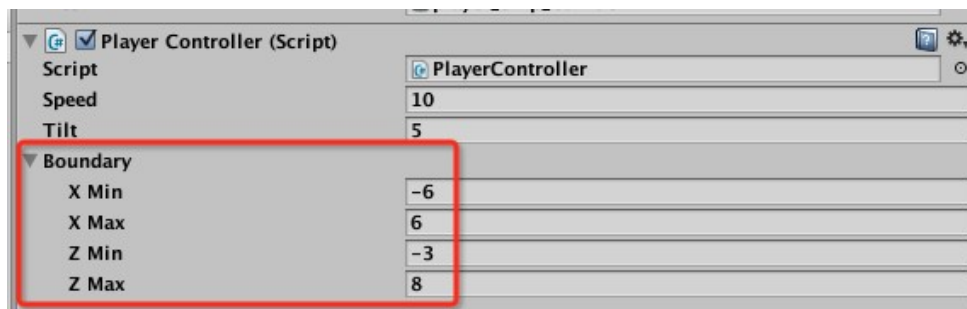
        Vector3 movement = new Vector3 (moveHorizontal, 0.0f, moveVertical);
        GetComponent<Rigidbody> ().velocity = movement * speed;

        GetComponent<Rigidbody> ().position = new Vector3 (
            Mathf.Clamp (GetComponent<Rigidbody> ().position.x, boundary.xMin, boundary.xMax),
            0.0f,
            Mathf.Clamp (GetComponent<Rigidbody> ().position.z, boundary.zMin, boundary.zMax)
        );
    }
}
```

完整的代码

- 首先**序列化**了一个**Boundary**类，用来保存飞船在X和Z轴上的范围，也就是xMin、xMax、zMin、zMax
- 然后我们定义了一个变量**boundary**
- 最后我们获取飞船的位置属性，然后通过**Mathf.Clamp**来确保飞船在X和Z轴上的范围处于我们设定的值内

将xMin、xMax、zMin、zMax整合到Boundary类里面，是方便分类管理公共变量，如下图：可以通过小三角来收起/展开，这样让公共变量更加整洁



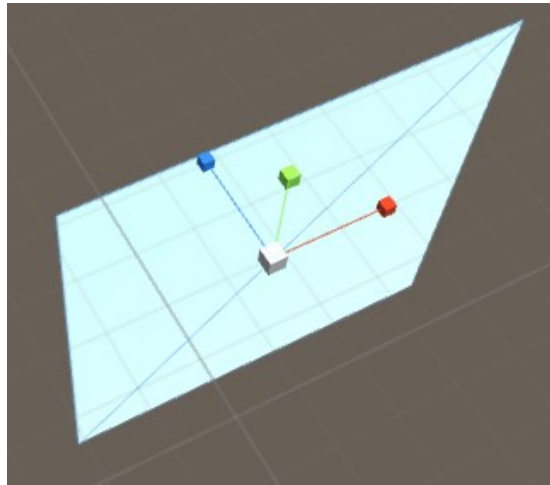
Paste_Image.png

写完代码后，我们保存一下，然后回到Unity编辑器，将xMin、xMax、zMin、zMax的值分别设置为（-6，6，-3，8），然后运行游戏测试一下，就可以发现，现在飞船无法跑出屏幕外了，Nice！

四、添加游戏背景

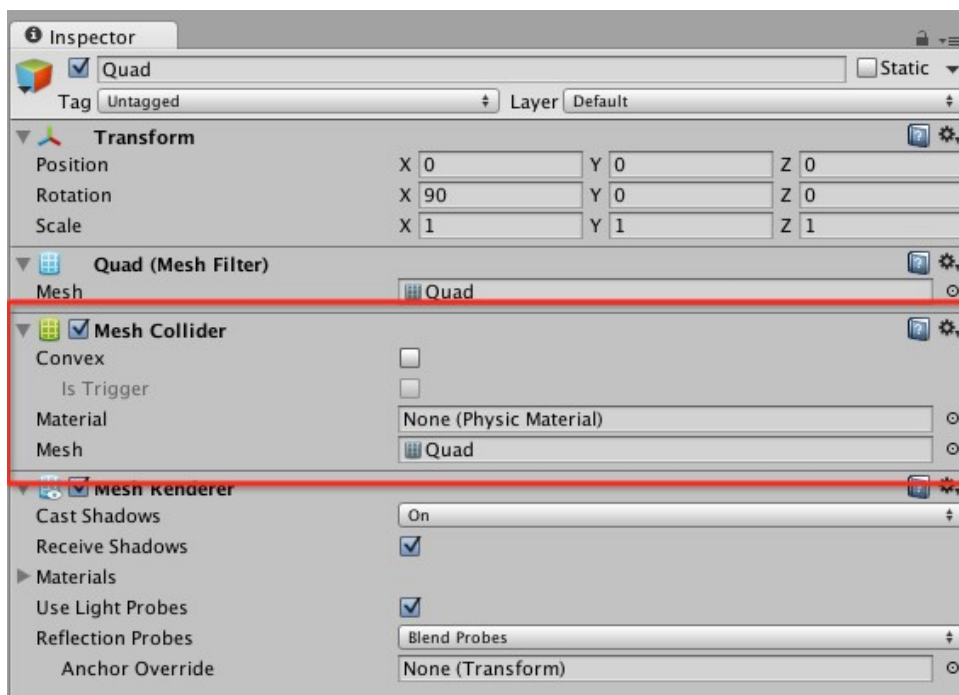
和官方视频不同，我们这边到这一步才添加游戏背景，原因是游戏背景和后面马上要讲到的子弹，都用到了同一个GameObject——**Quad**，所以两个东西放到一起来讲

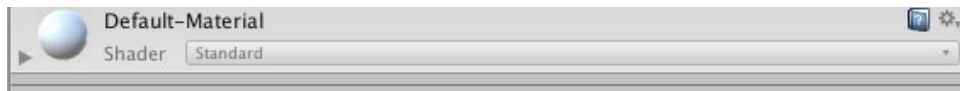
首先我们新建一个**Quad**对象，然后进行两个标准动作，将名称命名为**BackGround**，新建成功后，我们在**Game**视窗中可以看到，没有任何变化，为什么呢？我们回到**Scene**视图来看一下，Quad到底是个什么东西



Quad

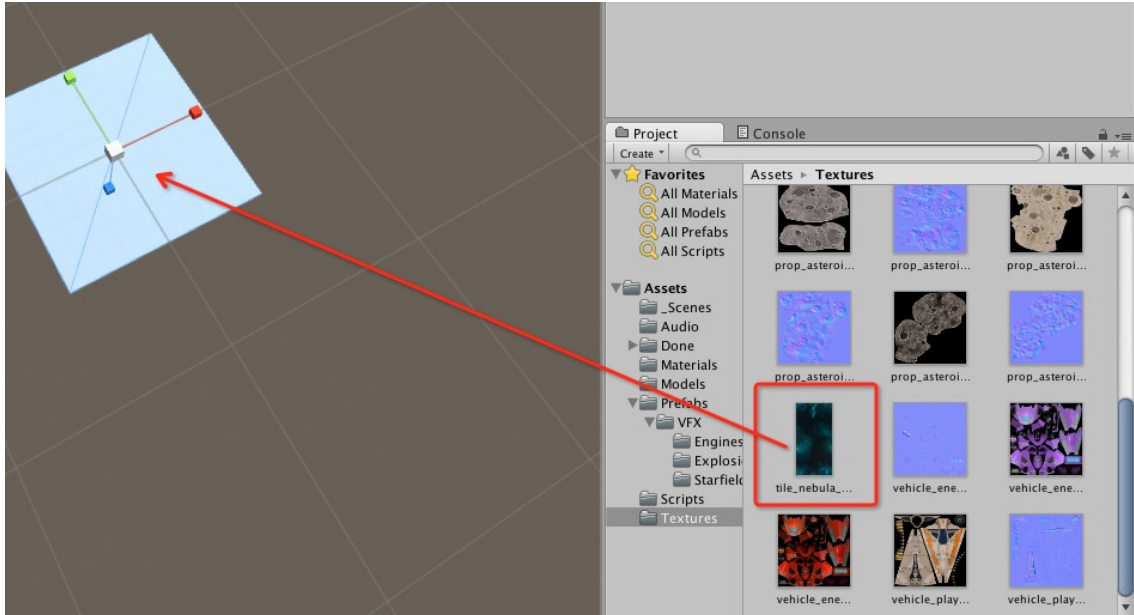
我们可以看到，Quad就是一个类似平面（Plane）的对象，它使垂直与摄像机的，所以我们看不到他，所以我们需要将它X轴上旋转90°，然后我们就可以在**Game**视窗中看到了





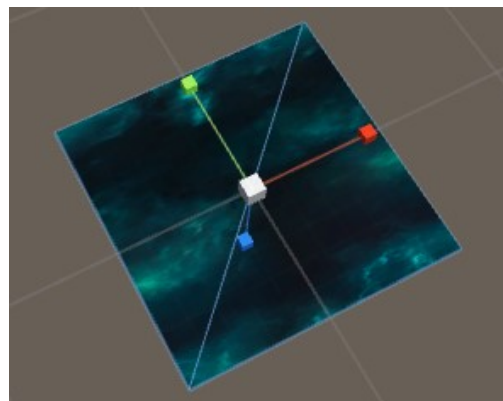
Paste_Image.png

我们观察一下**BackGround**的**Inspector**属性，由于我只是做背景用，所以**Mesh Collider**我们根本不需要，这里我们就将该组件给移除掉



Paste_Image.png

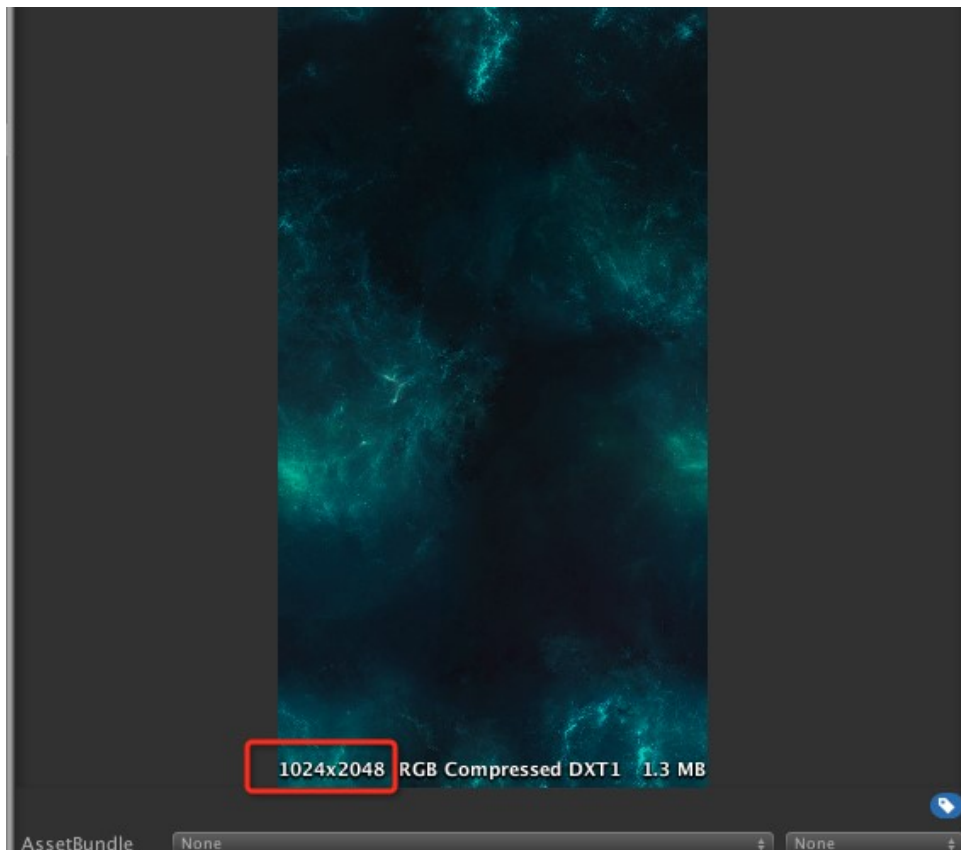
接下来我们将背景图片资源，直接拖入**BackGround**里面，Unity自动为我们生成了一个材质，此时我们可以看到**BackGround**变成了如下的样子：



Paste_Image.png

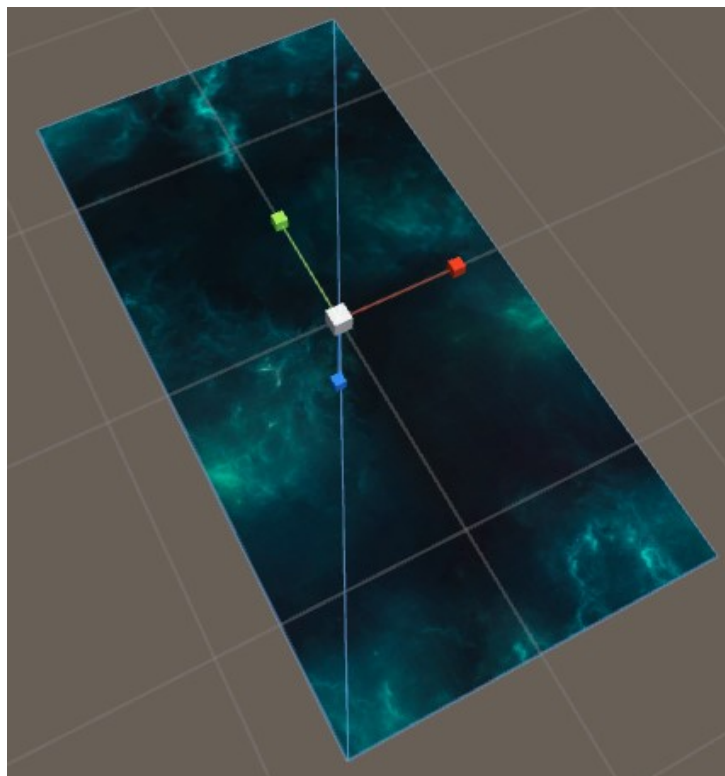
这就是我们的背景，额，作为一个背景，它好像还是有很多问题，太小、太暗，比例失调，所以首先第一步，我们就是将它的大小调整到合适的尺寸，我们选中背景图片资源，观察一下他的信息





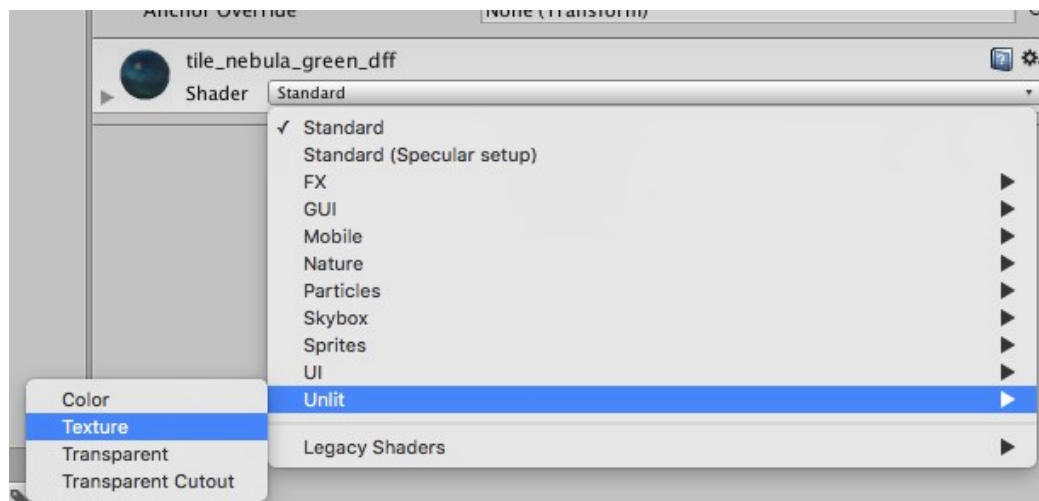
背景图片资源信息

我们可以看到，背景图片的尺寸为1024x2048，为了让图片比例不会失调，我们需要将*BackGround**的X设置为Y一半，我们将XY分别设置为（15，30）



Paste_Image.png

我们可以看到，现在**BackGround**的比例已经正常了，大小也合适，但是图片还是太暗了，这是由于我们选中的材质渲染模式有关系，我们将**Shader**修改为**Unlit/Texture**后，图片颜色就会变得正常了



Shader修改为Unlit/Texture

最后一步，作为背景，我们发现它有一部分将飞船给遮挡了



飞船被遮挡了

所以我们需要将背景往下移动一些，而且由于我们的摄像机是选用的正交模式，所以背景下移并不会改变他的显示效果，我们将**BackGround**的Position Y设置为-10，这样背景就不会在挡住飞船了

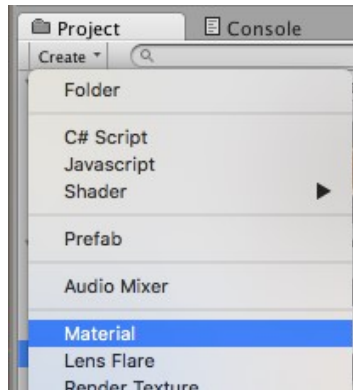
五、制作子弹

为了视觉效果和逻辑分离，我们首先新建一个空的**GameObject**，命名为**Bolt**，然后继续新建一个**Quad**，命名为**VFX**，将VFX拖入到Bolt下面，然后我们选中VFX，把它的Rotation X设置为90，同样的，作为Quad，我们也需要像背景一样，将一个图片显示在上面，但是，这里我们和设置背景图片不一样，我们使用一种新的办法来设置，首先我们新建一个材质

新建材质

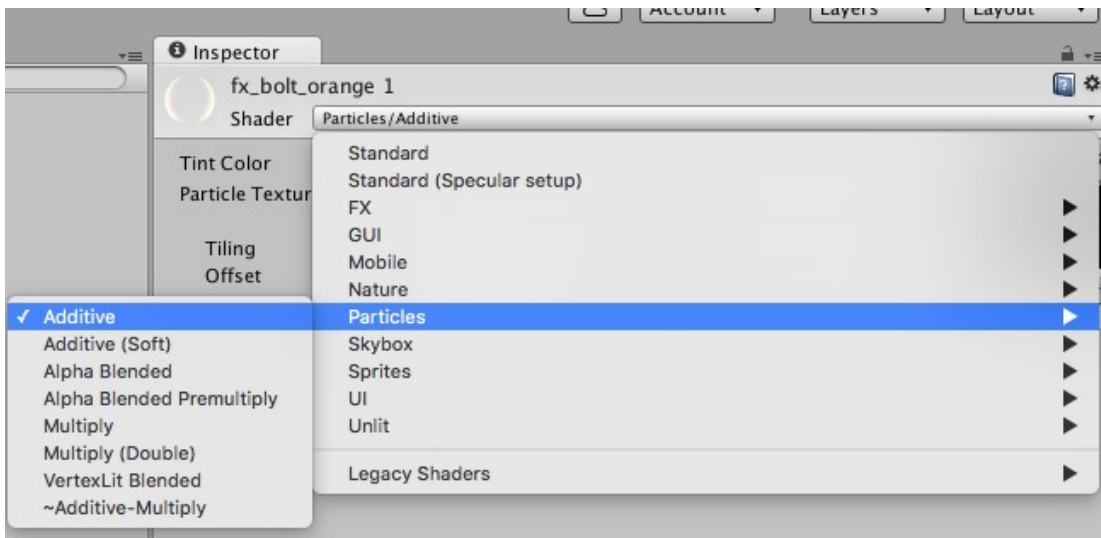
新建材质

首先我们选中**Create->Material**，创建一个新的材质，命名为**fx_Bolt_orange**



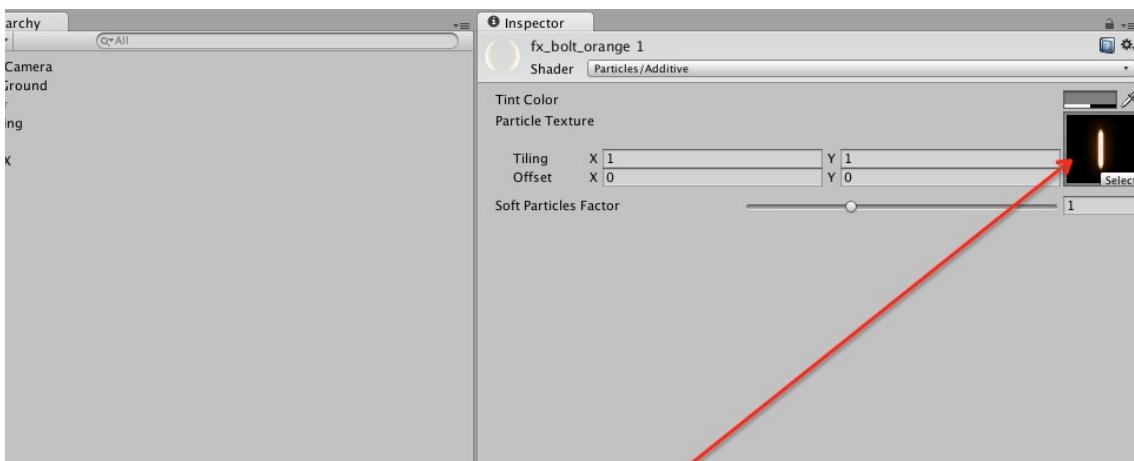
新建材质

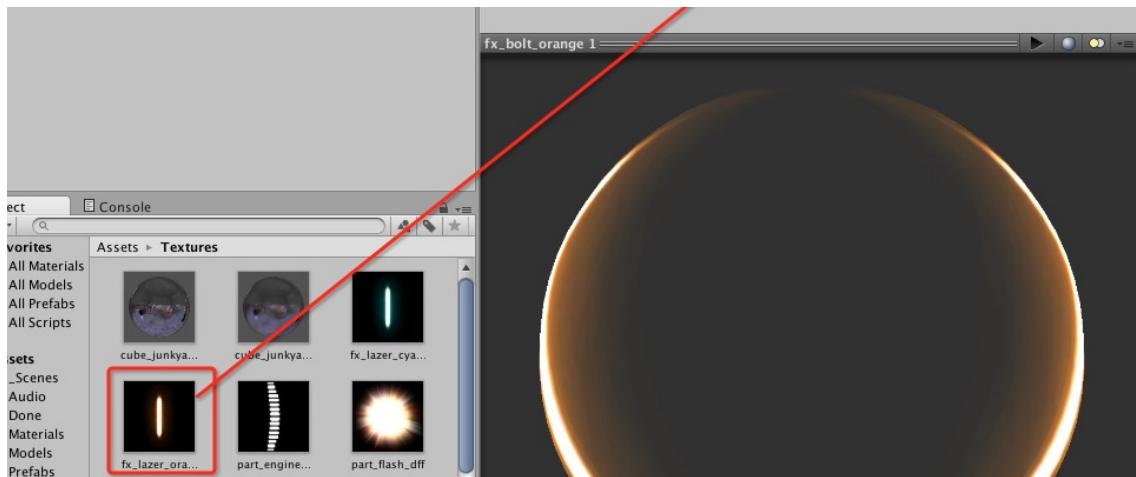
然后我们选中刚刚新建的材质，修改它的**Shader**模式为**Particles/Additive**



设置Shader

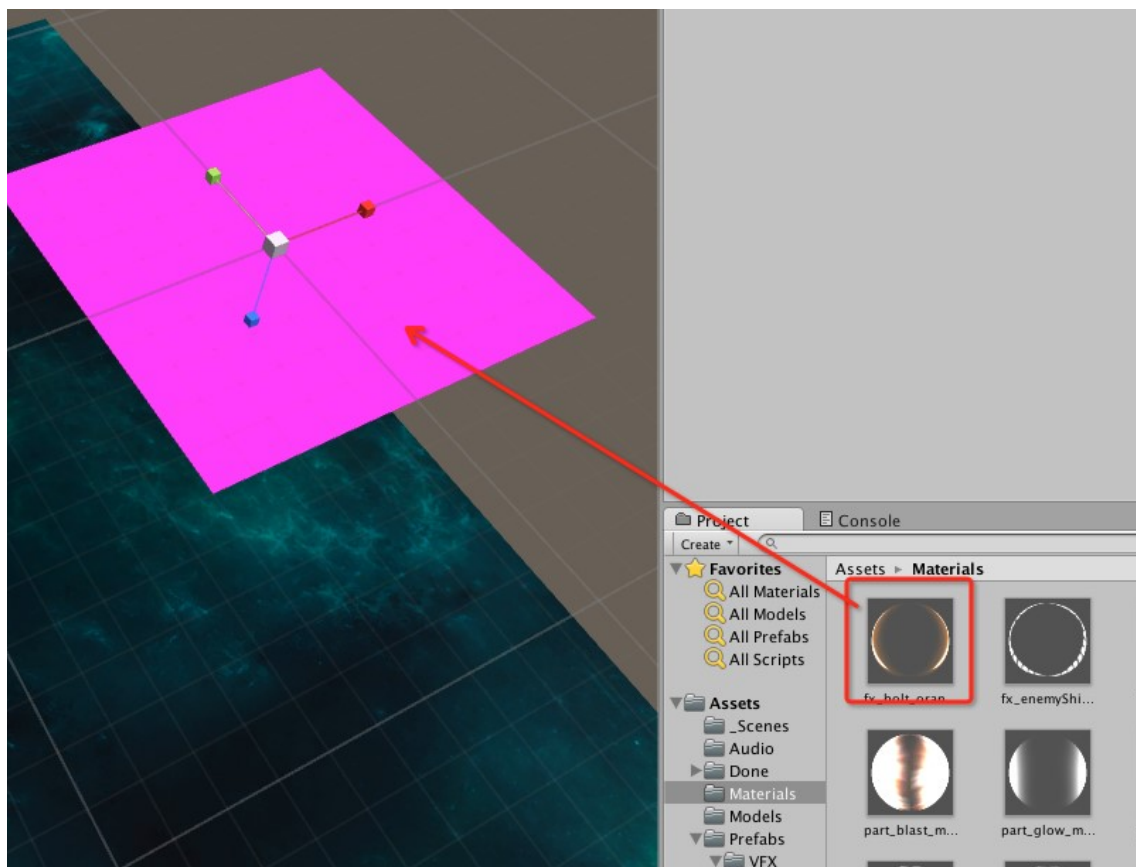
然后将子弹的图片和材质关联起来，如下图：





关联子弹图片

最后我们将我们做好的材质拖入VFX中



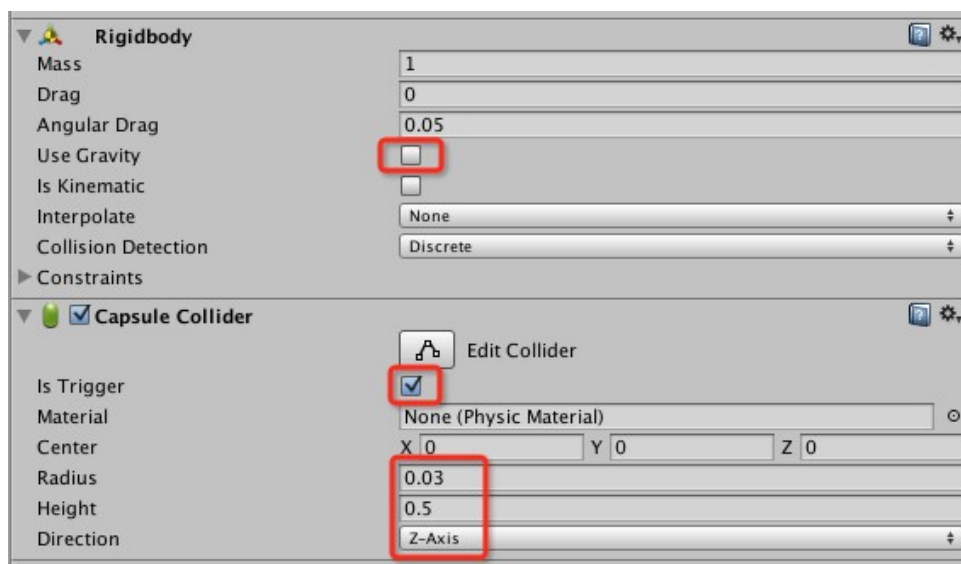
关联VFX

我们便可看到子弹的效果做好了！



给子弹添加碰撞

一开始我们说了要视觉效果和逻辑分离，所以我们移除VFX中的**Mesh Collider**组件，然后给**Bolt**添加刚体和胶囊碰撞（Capsule Collider），同时按下图设置他们的属性：



刚体和胶囊碰撞的属性设置

让子弹飞一会儿

到这里，子弹的视觉效果和物理属性我们都已经设置好了，下一步我们让子弹可以在发射后往前移动，如果子弹不能移动那还怎么打到人呢？

所以我们给Bolt添加一个新的脚本，命名为**Mover**，然后打开并编辑它，如下图：

```
4 public class Mover : MonoBehaviour {  
5     public float speed;  
6  
7     // Use this for initialization  
8     void Start () {  
9         GetComponent<Rigidbody> ().velocity = transform.forward * speed;  
10    }  
11 }
```

完整的代码

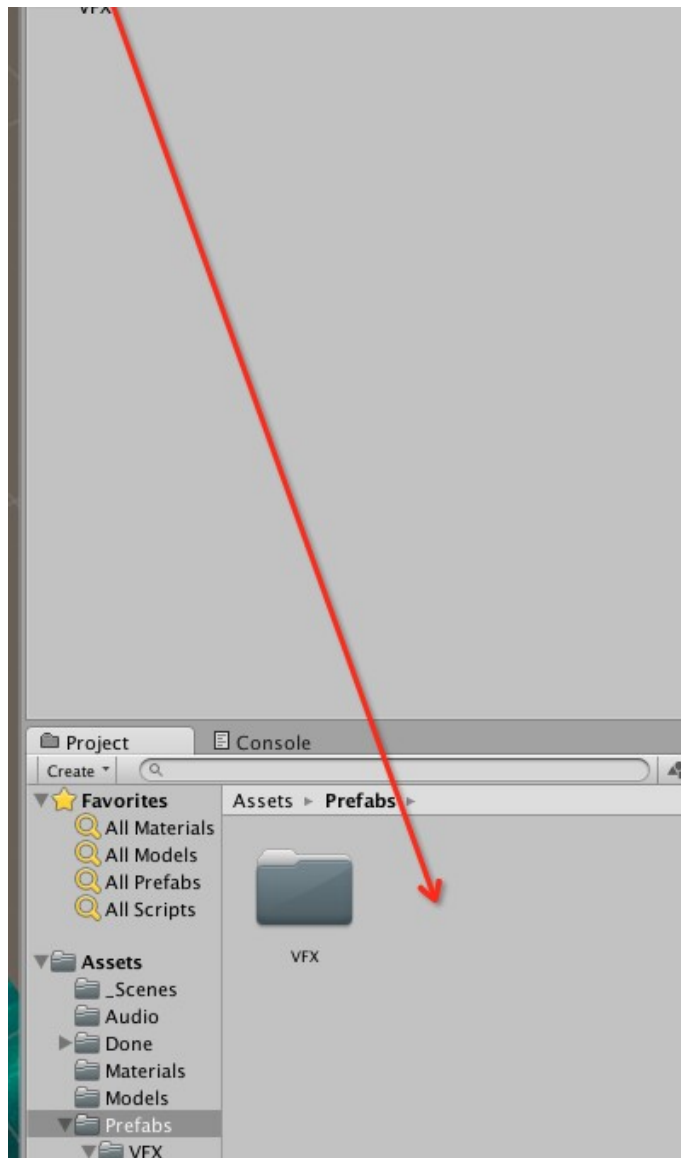
- 首先我们定一个公共变量**speed**用来调节子弹飞行的速度
- 然后我们在子弹生成时赋予它一个往前的速度

写完代码后，我们保持一下，然后回到Unity编辑器，将**speed**速度设置为20

制作成Prefab

做完子弹的所有准备工作后，我们将Bolt制作成一个Prefab（预制件），方便以后使用

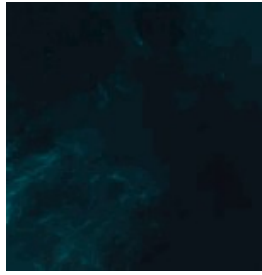




制作成Prefab

最后我们边可以运行一下，看到子弹飞出去的效果了





子弹飞行效果

六、飞船发射子弹

现在我们有了飞船，有了子弹，下一步就应该把它们联合起来，让飞船可以发射子弹，那么开始之前，我们思考一下，要实现飞船发射子弹的功能，我们需要一些什么？

- 首先我们要知道子弹从哪里发射
- 然后我们需要检查用户按键，来接受发射信号
- 然后我们需要设置子弹发射的频率
- 然后我们还需要知道如何生成一个子弹

我们一个个来看

设置子弹发射点（Shot Spawn）

在实现子弹发射之前，我们将场景中现有的Bolt删除，因为我们不需要他了

首先我们新建一个空的GameObject，命名为**Shot Spawn**，作为飞船的一个武器挂载点，确保所有的子弹都是从这个点出发，然后我们把**Shot Spawn**拖入到Player下面，并调整**Shot Spawn**的位置到合适的点，本例中置为飞船前方就可以了，如下图：



发射点的位置

子弹发射的频率

实现反射功能的代码

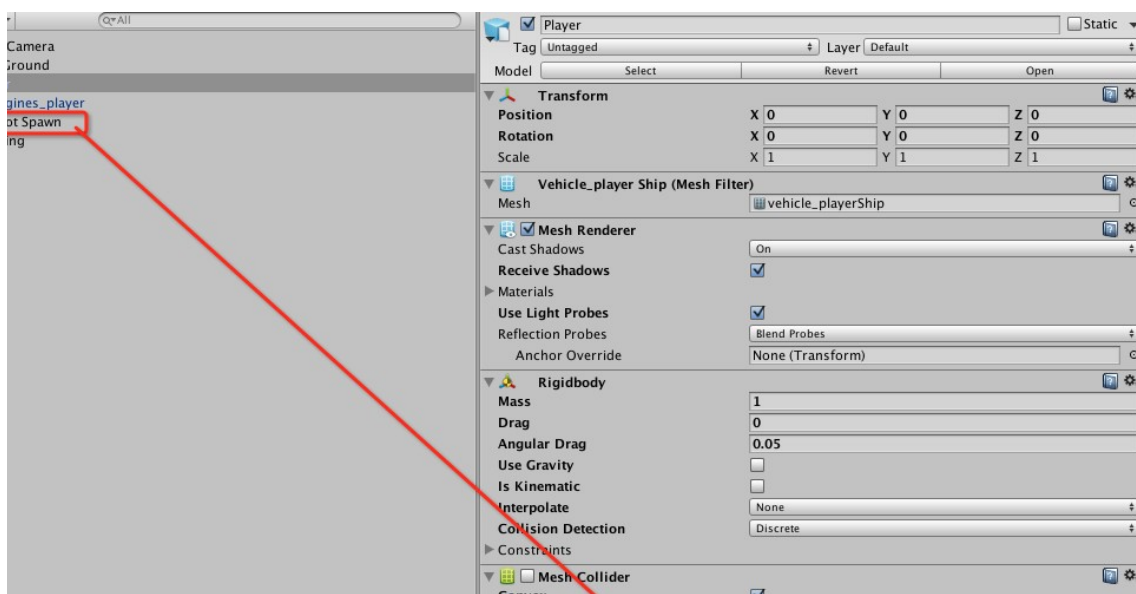
然后我们打开PlayerController脚本，新增以下代码：

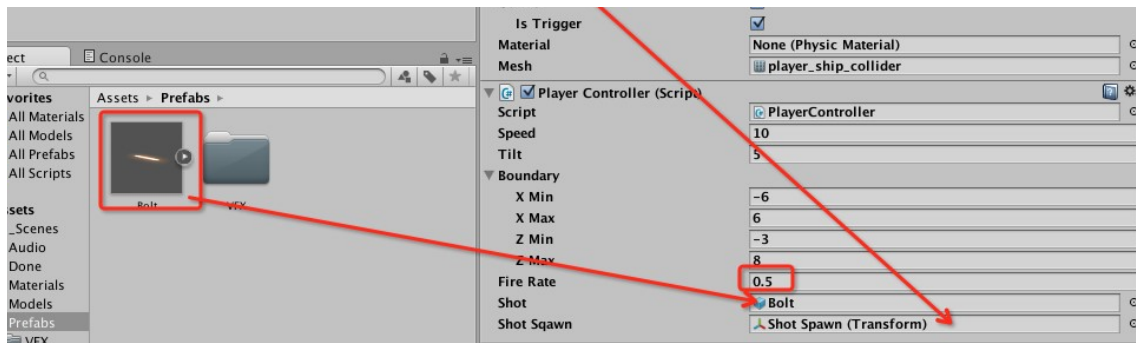
```
9 public class PlayerController : MonoBehaviour {
10     public float speed;
11     public float tilt;
12     public Boundary boundary;
13
14     private float nextFire;
15     public float fireRate;
16     public GameObject shot;
17     public Transform shotSqaun;
18
19     // Update is called once per frame
20     void Update () {
21         if (Input.GetButton ("Fire1") && Time.time > nextFire) {
22             nextFire = Time.time + fireRate;
23             Instantiate(shot, shotSqaun.position, shotSqaun.rotation);
24         }
25     }
26 }
```

完整的代码

- 首先我们定义一个私有变量**nextFire**，用来记录下一发子弹的时间
- 然后我们定义一个公共变量**fireRate**，用来控制子弹发射的间隔时间
- 接着我们定义一个公共变量**shot**，用来保存我们发射的子弹Prefab，也就是告诉程序，我们发射的是哪个子弹
- 然后我们定义一个公共变量**shotSqaun**，用来保存子弹的发射点，也就是告诉程序，子弹从哪里发射出去
- 接着我们在Update函数中，判断用户是否有按下**Fire1**键，这边的**Fire1**键就是键盘上的左Ctrl键和鼠标左键，同时当前时间超过我们下一发子弹所需的时间，换句话说，这个if的作用就是确保我们武器没有在冷却时间时按下发射键，才会发射子弹
- 当if条件通过后，我们先记录下一发子弹的时间，然后通过Instantiate方法生成一发子弹，他有3个参数，一个是发射的对象，一个是发射对象的position，一个是发射对象的rotation

写完代码后，我们保存一下，然后回到Unity编辑器，并分别设置我们刚刚定义的几个公共变量，如下：





设置公共变量

所有操作做完后，我们边可以运行游戏，看看实际效果了！



发射子弹效果

下集内容

在下一集，我们将学会：

- 如何回收子弹
- 如何创建障碍物
- 如何通过代码创建无尽波数的障碍

- 如何实现子弹击中障碍物破碎功能